Synthetic data: manufacturing the tokens you cannot find

Why generation comes before training, how the frontier rephrases the web and teaches itself facts, the scaling laws that say how much to make, and the quality control that keeps a factory from poisoning its own supply

---

Your SFT plan (Chapter 23) calls for 200,000 high-quality instruction–response pairs in Spanish and Portuguese. The open datasets are English translated badly, the vendors quote dollars per pair with six-week lead times, and the annotation budget covers a twentieth of the need. Meanwhile the strongest model in your cluster drafts a competent pair in two seconds for a hundredth of a cent. Two floors down, the pre-training team has a different version of the same problem: Chapter 10's pipeline produced 1.3T curated tokens for a model whose compute budget wants 2T, and Chapter 10's data-constrained law says the fourth epoch is the last free one. In both rooms the move with the bad reputation — training on model outputs — is now what the frontier runs on. Phi-4 pre-trained on about 400B tokens of curated synthetic text and mostly synthetic post-training data[1]; Nemotron-4's alignment pipeline was 98% synthetic[2]; Nemotron-CC added 1.9T tokens of rephrased web to Common Crawl[3]; Qwen3 pre-trained on trillions of tokens generated by its own specialist models[4]; and Chapter 26's reasoning distillates were built from 800K sampled traces. The question stopped being whether to manufacture data. This chapter is about how to manufacture it so that quality goes up, how much of it to make, and the well-documented ways it goes down.

One framing organizes everything. Synthetic data is a factory with three stations: **generation** (what raw material, shaped how), **quality control** (filtering, verification, rejection — where most of the value is created), and **compliance** (licenses, provenance, and the contamination ring of Chapter 11). Teams fail at the second and third stations far more often than at the first, because generation is the fun part. The chapter walks the stations in order, with the 2025–2026 results that turned synthesis from a post-training trick into a pre-training lever: rephrasing at trillion scale, a model that writes its own study plan, and the first scaling laws that say when synthetic tokens stop paying.

27.1 Why generate before you train

Start with what synthetic data is for, because "more tokens" is the least interesting answer. There are three gaps a training set can have, and synthesis closes each differently. The **token gap** is the one Chapter 10 ended on: the curated

web is finite, repetition beyond about four epochs stops paying ($D_{\text{eff}} = U + UR^\star(1 - e^{-R/R^\star})$, $R^\star \approx 15$), and every lab now plans for the point where the model's compute wants more good tokens than exist. The **coverage gap** is subtler and bigger: a Spanish-first model's web has 4.6% Spanish and almost no Spanish mathematics, chemistry, or code comments; a domain model's corpus has the facts but not the question-shaped text that teaches a model to *retrieve* them. The **format gap** is the post-training one: the base model knows the answer and has never seen a conversation, a reasoning trace, or a preference. Figure 27.1 lays the three out with the generation pattern that fits each, and the rest of the chapter follows those columns.

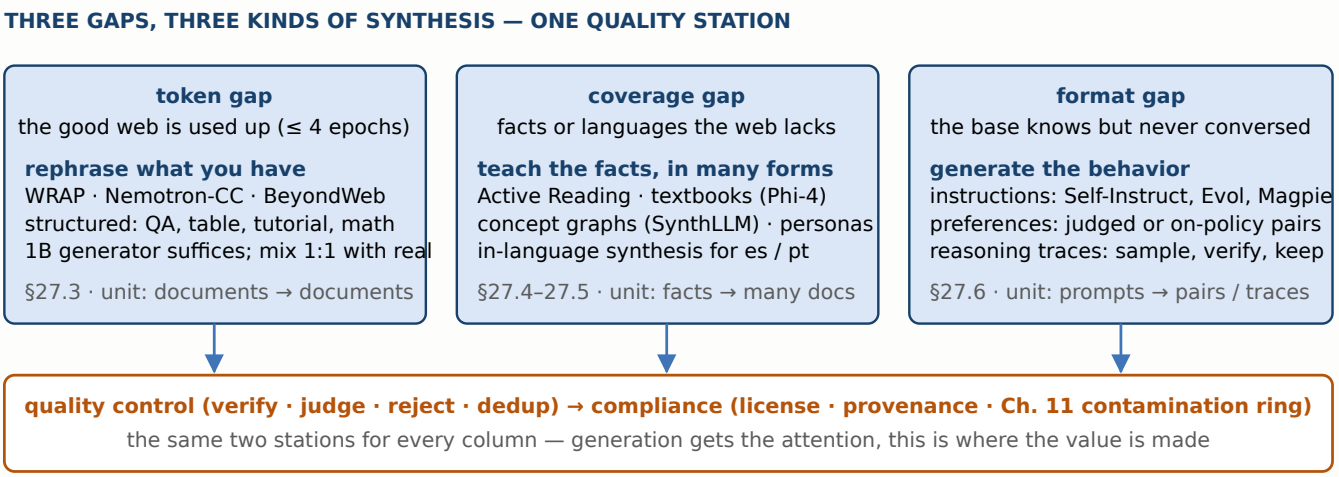


Figure 27.1 — Which gap are you filling? The generation pattern follows from the gap; the quality and compliance stations do not change. A team that cannot name its gap generates the wrong thing well.

The deepest lesson of the 2025 pre-training work is one sentence from the BeyondWeb report: synthetic data must be *deliberately designed to beat real data* — it is not free quality[5]. The naive experiment, "have a strong model write more of the same," reliably produces tokens that are fluent, on-distribution, and worth less than the originals, because the generator's output is a lossy sample of what it already knows; the model you train on it learns a narrower distribution with the same ceiling. What makes synthetic data worth more than its source is *transformation*: changing the form of information so that the student sees the same fact as a question, a table, a tutorial, a dialogue, a worked problem — forms that are rare on the web and that the student will be asked to produce. That is why every method in this chapter is a transformation with a seed, not a generation from nothing, and why the one method that generates from nothing (Magpie, §27.6) works by extracting what a tuned model has *already learned* rather than inventing it.

27.2 The design space: seeds × prompts × verifiers

Every generation recipe answers three questions, and the twenty named methods in the literature are points in a space with three axes. **Where does variety come from?** From a seed: a web document to be rephrased, a fact to be taught, a persona to write as, a concept pair to combine, a prompt to answer. Diversity is manufactured at the seed, not at the sampling temperature — a generator asked the same thing a million times at temperature 1 produces a million paraphrases of one idea, which is why the 1-billion-persona idea exists (every persona is a different seed, and a 7B model fine-tuned on 1.07M persona-generated math problems reached 64.9% on MATH)[6], and why Active Reading asks the model to write *different strategies* for each document before writing anything. **What does the generator do**

with the seed? That is the prompt, and the April 2026 Hugging Face study that varied prompts, generators, and sources over more than a trillion generated tokens found prompt design to be the dominant factor — a structured "rewrite as math problems" prompt beat the best paraphrase prompt, which beat raw DCLM, while scaling the generator from 1B to 27B often made results *worse*[7]. **Can the output be checked?** Math and code have verifiers; facts have the source document; conversations and essays have only a judge. The verifier axis decides the quality station's design (§27.6) and, in the end, how far you can trust the data.

Table 27.1 — The design space, by seed. Each row is one family; the "gap" column is Figure 27.1's. Numbers are the headline results from the cited sources.

SEED → PROMPT	GAP	VERIFIER	REPRESENTATIVE RESULT
Web document → rephrase (paraphrase, QA, Wikipedia style, structured formats)	token	source document (faithfulness)	WRAP: ~3× pre-training speed-up, >10% perplexity gain[8]; BeyondWeb: 7.7× faster than open web at 8B[5]; FinePhrase-Table: +3.4 over DCLM[7]
Document → self-written study strategies → apply each	coverage (facts)	source document; QA grader	Active Reading: 66% on SimpleWikiQA vs. 26% for paraphrase and 48% for synthetic QA, same 8B model[9]
Topic / concept graph → textbook, lecture, problem set	coverage	judge; verifier for problems	Phi-4: ~400B synthetic tokens, 50 generators[1]; Cosmopedia: 25B tokens, 30M documents[10]; SynthLLM: rectified scaling law, plateau ≈ 300B[11]
Persona → anything ("as a Quechua-speaking nurse, write…")	coverage, format	judge; verifier if math/code	1B personas; 64.9% MATH from 1.07M generated problems[6]
Seed instructions → evolve / self-instruct; or nothing → Magpie	format	judge; dedup	Self-Instruct: 175 seeds → 52K[12]; Magpie: 4M pairs from the chat template alone[13]
Prompt → sample N responses → verify → keep (rejection sampling)	format (reasoning)	exact / execution / reward model	DeepSeek-R1: ~800K kept traces[14]; OpenThoughts3: 1.2M, source quality > diversity (Ch. 11)

Two general facts about the space before the sections that walk it. First, **the generator does not need to be frontier.** Across the rephrasing studies a 1B–3B generator captures most of the gain (BeyondWeb: 1B +1.8, 3B +3.3, 8B +3.7 points over the baseline — diminishing after 3B[5]; FinePhrase: Gemma-3 1B beat Gemma-3 27B on the downstream average while costing 5–10× less[7]), and Active Reading found that data generated by the 8B model itself beat data generated for it by a 70B model (66.3 vs. 62.3)[9]. The reason is that the value is in the transformation, and a small model transforms as well as a large one; the large one adds knowledge the student may not be ready to absorb. Second, **the cost is inference, so Chapter 13's economics apply.** A generator with N_g parameters writing T tokens costs about

$2N_gT$ FLOPs plus the prefill of its prompts; at the throughput the FinePhrase team reports for a 1.7B rephraser with speculative decoding — about 9,200 output tokens per second per GPU — a trillion tokens is roughly

$$\frac{10^{12}\ \text{tokens}}{9{,}200\ \text{tok/s/GPU} \times 86{,}400\ \text{s/day}} \approx 1{,}260\ \text{GPU-days},$$

and their 486B-token release took about 612 GPU-days, consistent with that rate[7]. Against the 66K GPU-hours (2,750 GPU-days) of the 7B/2T training run, generating half the corpus is a quarter of the training cost — real money, and an order of magnitude less than the 27B-generator version of the same run, which is why "pick the smallest generator that passes the quality station" is the rule.

27.3 Rephrasing the web at trillion scale

The pre-training version of synthesis starts from the observation that the web's problem is form, not content. Most documents that survive Chapter 10's filters contain something worth knowing, written in a way that is a poor teacher: buried in navigation, hedged, repeated, in a register the model will rarely be asked to produce. **WRAP** (Rephrasing the Web) was the first systematic demonstration that asking an instruction-tuned model to rewrite each document — "like Wikipedia," "as a Q&A," "for a child," "in high-quality English" — and training on the rewrites mixed with the originals gave about a 3× speed-up in pre-training and more than 10% lower perplexity on held-out text at matched tokens[8]. The rewrite preserves the content and changes the distribution of forms toward what the evaluations ask for; it is Chapter 11's relevance lever (BETR) applied by editing documents instead of selecting them.

Three scaled-up results fix the recipe. **Nemotron-CC** rephrased low-quality crawl into usable text and high-quality crawl into diverse QA pairs, distilled summaries, and knowledge lists, adding 1.9T synthetic tokens; at 1T training tokens its mix beat DCLM by 5.6 MMLU points on 8B models, and the synthetic portion is what made the *low-quality* crawl worth keeping[3]. **BeyondWeb** ran the ablations as a science project: 14 benchmarks, 1B/3B/8B students, 180B-token runs. Its rephrased data beat Cosmopedia by 5.1 points and Nemotron's synthetic set by 2.6 at 8B; it matched the open-web baseline's 180B-token score at 23B tokens (7.7× faster); a 3B model trained on it beat an 8B model trained on Cosmopedia at the same budget; and — the finding that matters for a budget — it kept improving at 1B out to roughly 50× the Chinchilla-optimal token count, where the other datasets had overfit[5]. **FinePhrase**, the Hugging Face study of April 2026, is the most complete map: twelve prompts × two generator families at six sizes × six architectures × four source/mix-in settings, each run rephrasing 10.5B tokens and training a 1.2B model on 21B. The ordering of effects was prompt ≫ generator family > generator size, with four new *structured* formats (rewrite as a math problem set, a table, an FAQ, a tutorial) beating every paraphrase-style prompt, and the released 486B-token FinePhrase-Table set scoring 3.4 points above DCLM and 3.6 above Nemotron's synthetic high-quality set at 30× lower generation cost than a frontier-generator pipeline[7]. Figure 27.2 puts the comparable numbers side by side.

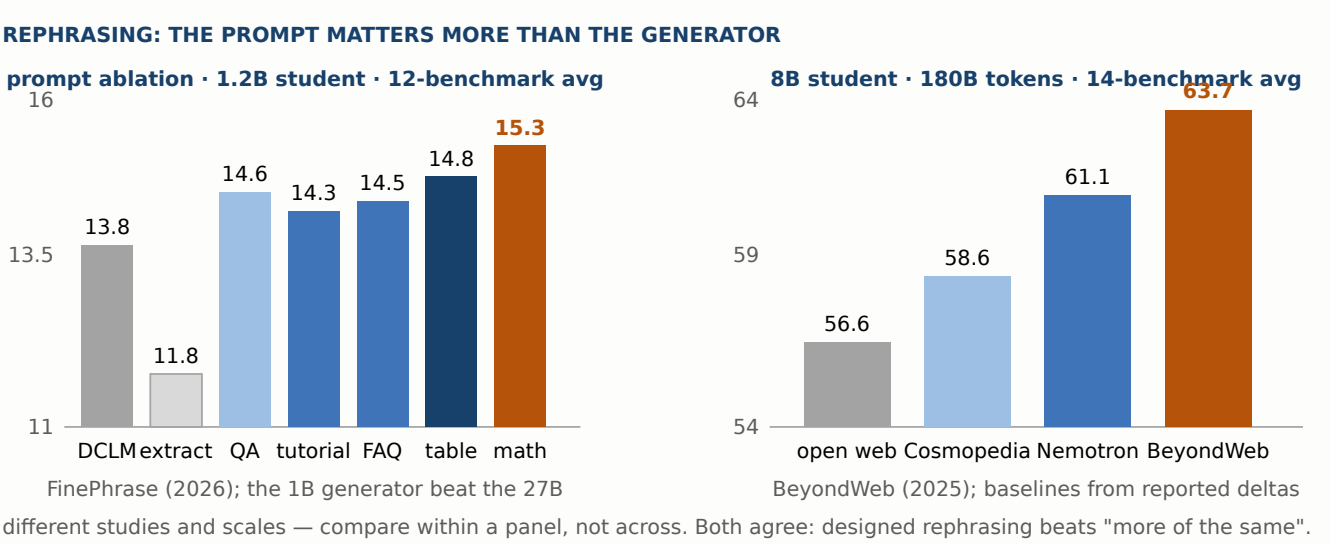


Figure 27.2 — What rephrasing buys. Left: FinePhrase's prompt ablation at fixed generator and budget — the spread across prompts (3.5 points) is larger than the spread across generator sizes. Right: BeyondWeb's 8B comparison across public synthetic sets. In both, the winning data is a *transformation* of the web (tables, problems, conversations), not a paraphrase of it.

Four rules fall out of the three studies, and they are the ones a team should copy. **Always mix.** Synthetic-only training lost 2–3 points on every prompt in FinePhrase; mixing with the original web text restored commonsense and language-understanding scores, and the choice of mix-in corpus mattered more than the quality of the source being rephrased[7]. BeyondWeb's ablations used 50:50. **Rephrase the mediocre, not only the best.** Rephrasing low-quality source with a high-quality mix-in came within 1.8 points of rephrasing high-quality source[7], and Nemotron-CC's low-quality rewrites are what rescued the bottom of its crawl — a cheap way to raise Chapter 10's funnel yield. **Diversity is the thing that keeps scaling.** BeyondWeb's data kept improving where Cosmopedia and Nemotron's overfit, and FinePhrase found that a generator which followed the format too rigidly (Qwen3) produced "template collapse" and trained worse than a messier one (SmolLM2)[57]. **Style matching is bounded.** Raising the share of conversational text from 3.7% to 20% gained under a point and then saturated[5]. The generation loop itself is a vLLM job:

```
# rephrase FineWeb-Edu sample-10BT into a structured format with a small generator
(vLLM)
from vllm import LLM, SamplingParams
gen = LLM("HuggingFaceTB/SmolLM2-1.7B-Instruct", dtype="bfloat16",
max_model_len=4096)
FORMATS = {
  "table":    "Rewrite the key facts of the text as Markdown tables with short
captions.",
  "math":     "Write 3-5 self-contained math problems using quantities and
relations "
              "from the text, each with a worked solution.",
  "tutorial": "Rewrite the text as a step-by-step tutorial a beginner could
follow.",
  "faq":      "Rewrite the text as an FAQ of 6-10 Q/A pairs covering all its
information.",
}
sp = SamplingParams(temperature=0.7, top_p=0.95, max_tokens=1024)  # diversity >
```

```
fidelity
def batch(docs, fmt):
    prompts = [f"{FORMATS[fmt]}\n\nText:\n{d['text'][:6000]}\n\nOutput:" for d in
docs]
    for d, o in zip(docs, gen.generate(prompts, sp)):
        yield {"text": o.outputs[0].text, "source_id": d["id"], "format": fmt,
               "generator": "SmolLM2-1.7B-Instruct", "prompt_version": "v3"}  #
lineage
# train on synthetic:real ~ 1:1; verify a sample against the source (§27.6) before
scaling
```

27.4 Teaching facts: Active Reading

Rephrasing raises average quality. It does not solve the problem that defines the coverage gap: a fact that appears once, in one document, is almost never learned. Language models learn facts by seeing them many times in many contexts — the same mechanism that makes Chapter 10's four-epoch limit a limit — and most of what a domain corpus contains appears once. The 2024 remedy was to generate paraphrases and question–answer pairs for each document; both help, both plateau early. **Active Reading** (Meta FAIR, ICLR 2026) found a better remedy by asking the model what a good student would do with the document, and the result reset the ceiling on how much a small model can memorize[9].

The method has two stages, both run by the model that will be trained. In **stage one** the model reads a document and writes a list of *learning strategies specific to that document* — "paraphrase each paragraph in plain language," "write the timeline as a quiz," "connect the mechanism to one I already know," "explain the contradiction in the third section" — the kinds of things a student does with a textbook and a web crawl never contains. A task-specific variant tells the model it is studying for a trivia competition, has it write questions that cover every piece of information, then write a general strategy per question. In **stage two** each strategy is applied to the document independently ("Here is a learning strategy: … Apply it to the following document"), producing a set of augmentations that differ in form because the strategies differ, which is where the diversity comes from. Figure 27.3 shows the pipeline next to the baselines it replaces.

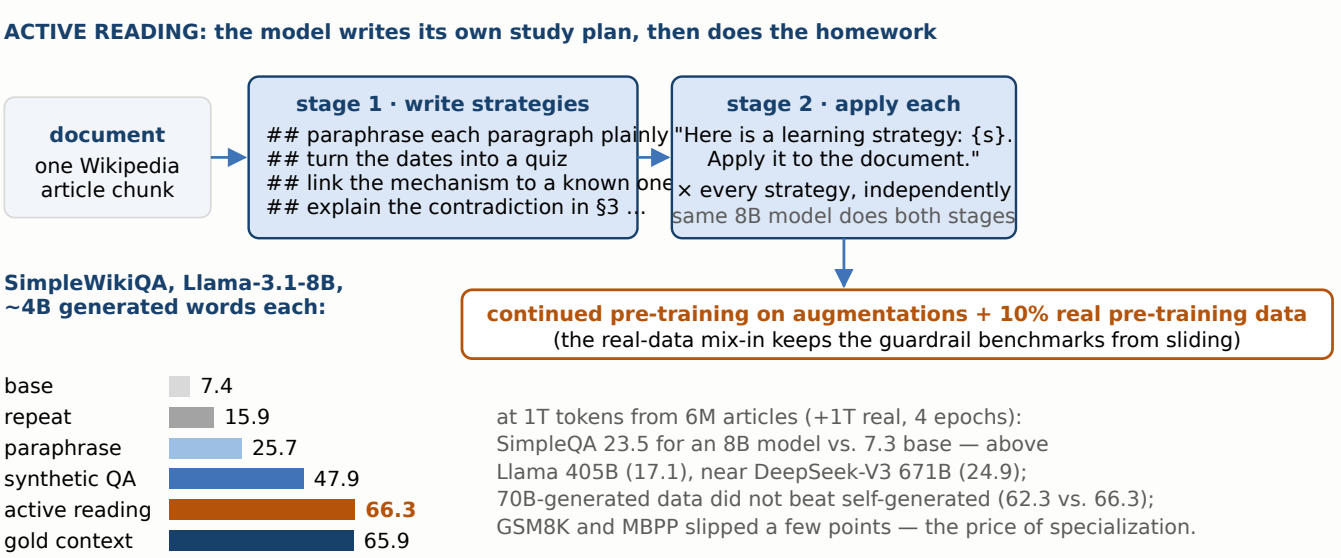


Figure 27.3 — Same model, same token budget, four ways to study. Paraphrasing and QA generation plateau; strategies the model wrote for itself keep improving with data and reach the accuracy the model gets when the article is in its context window. Numbers from the Active Reading paper's 8B experiments[9].

The numbers are the argument. On a Wikipedia-grounded subset of SimpleQA, an 8B model continued-pre-trained on ~4B generated words went from 7.4 (base) to 15.9 by repeating the articles, 25.7 by paraphrasing them, 47.9 by generating QA pairs, and 66.3 by Active Reading — essentially the 65.9 it scores with the article in context, which is the ceiling for a closed-book test[9]. Scaled to a trillion generated tokens from six million articles, mixed one-to-one with real data and trained for four epochs, the resulting 8B model scored 23.5 on the full SimpleQA against 7.3 for its base, above Llama 3.1 405B's 17.1[9]. The ablations teach three things a team will need. Diversity, not coverage, explains the gap — all four methods covered the test answers about equally, but Active Reading's outputs were far more varied by self-BLEU. Mixing in real pre-training data is not optional: without it, higher learning rates (needed at scale) degraded the general benchmarks and intermediate checkpoints gave fluent wrong answers. And the specialization has a price: the trillion-token model trailed its base on GSM8K (48.6 vs. 54.4) and MBPP (39.4 vs. 48.0), a reminder that this is a lever for *knowledge*, and that Chapter 25's reasoning and Chapter 26's distillation remain the levers for skill.

Why does it work? The paraphrase and QA baselines apply one fixed transformation to every document; the student sees each fact in two or three forms. Active Reading conditions the transformation on the document — a chronology gets a timeline quiz, a mechanism gets an analogy, an argument gets a steel-manned counterargument — so each fact arrives in the forms that are natural *for that fact*, and the set of forms is as large as the generator's repertoire of study habits. In the vocabulary of §27.2: the seed is the document, the prompt is written by the model per seed, and the verifier is the source document itself, against which every augmentation can be checked for faithfulness. For a Spanish-first model with a thin corpus of regional history, law, or medicine, this is the most direct route from "we have the documents" to "the model knows what is in them."

```
# Active Reading, two stages, run with the model you will train (vLLM)
STRAT = ("You are studying this document to learn and remember everything in it. "
         "Write learning strategies SPECIFIC to this document, one per line, each
"
         "starting with '## '.\n\nDocument:\n{doc}")
APPLY = ("Here's a learning strategy: {s}\nApply this strategy to the following "
```

```
        "document.\n\nDocument:\n{doc}")
def active_read(doc):
    plan = gen.generate([STRAT.format(doc=doc)], sp)[0].outputs[0].text
    strategies = [l[3:].strip() for l in plan.splitlines() if l.startswith("## ")]
    outs = gen.generate([APPLY.format(s=s, doc=doc) for s in strategies], sp)  #
independent
    for s, o in zip(strategies, outs):
        yield {"text": o.outputs[0].text, "strategy": s, "source_id": doc_id(doc)}
# mix: augmentations + ~10% real pre-training text; lr above SFT levels; watch
GSM8K/MBPP
# faithfulness: judge(text, doc) on a 1k sample; drop by strategy name if it
hallucinates
```

27.5 How much to make: scaling laws for synthetic tokens

Synthesis is not free and not unbounded, so the budget question needs a law. Chapter 7 gave the form for real data: loss falls as a power of tokens until the model saturates. The 2025 **SynthLLM** study asked whether synthetic tokens obey anything similar, generating them by extracting concepts from a corpus and recombining them through a graph (so that each document mixes several source concepts — a seed-side diversity mechanism, like personas) and training a family of models on increasing amounts[11]. Three findings. Synthetic data follows a *rectified* scaling law — the same shape as real data, with a floor and a knee — across model sizes; the gains plateau at roughly 300B synthetic tokens; and larger models need fewer of them, with an 8B model peaking near 1T synthetic tokens where a 3B model needed about 4T. Written in the rectified form,

$$L(D_s) \;=\; \frac{B}{(D_s + D_0)^{\beta}} \;+\; E,$$

where D_s is the number of synthetic tokens, D_0 accounts for what the model already knows before seeing them (the pre-training it started from), β is the fitted exponent, and E the floor the synthetic distribution cannot push below — the floor is the quantity that makes synthetic data different from real data, because it is set by the generator's own knowledge. The practical reading is in Figure 27.4: there is a region where synthetic tokens are worth nearly as much as real ones, a knee where they start to repeat the generator's distribution, and a floor beyond which only a better generator or new real data moves the loss. BeyondWeb's observation that its data kept improving at 50× Chinchilla while the plainer synthetic sets overfit is the same law with a different E: more diverse generation pushes the floor down and the knee out[5].

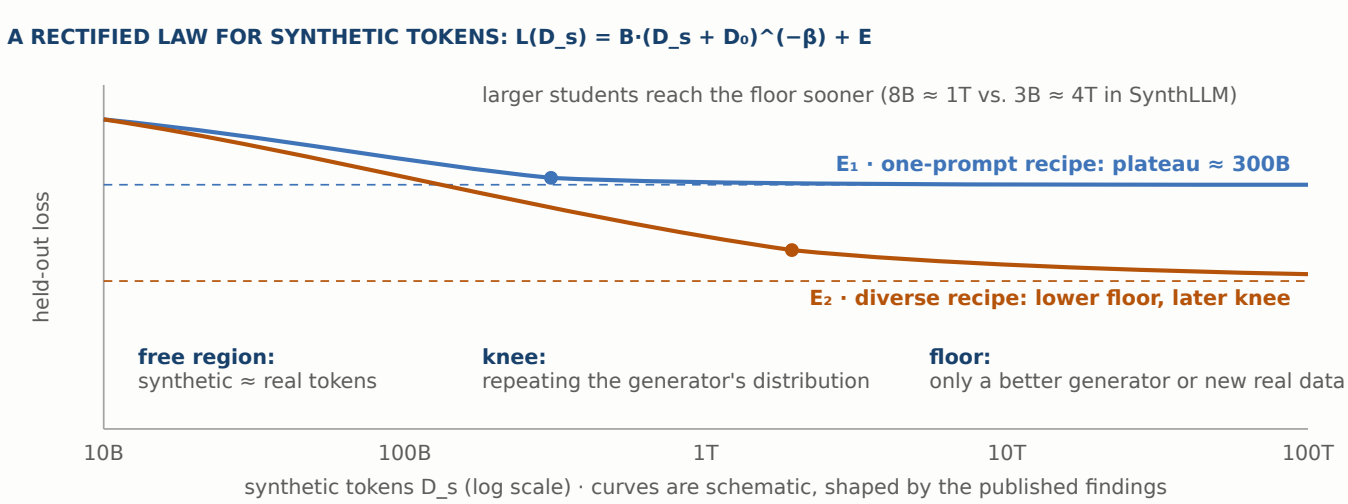


Figure 27.4 — Synthetic tokens have a floor, and diversity sets it. Both curves follow the rectified law; they differ in E and in where the knee falls. The budgeting rule is to generate up to the knee of your recipe — measured on a small model — and spend the remaining compute on raising diversity rather than on more tokens of the same kind.

How to use the law without a lab's budget: fit it on a proxy. Generate 50B tokens with your recipe, train a 300M–1B student on 5, 10, 20, and 50B of them (mixed one-to-one with real text), fit (B, D_0, β, E) by least squares on the held-out loss, and read off the knee. The fit costs a few hundred GPU-hours and tells you whether the next 500B tokens are worth generating or whether the recipe needs another prompt family first. Combine it with Chapter 10's data-constrained law and the two together answer the question the chapter opened with: how many real epochs, how many synthetic tokens, and in what mix, for a fixed compute budget.

27.6 Post-training synthesis and the quality station

The format gap is where synthetic data started, and where the quality station matters most, because the generator is producing *behavior* the student will imitate rather than facts it will absorb. Three kinds of artifacts, three generation patterns, one funnel.

Instructions. Self-Instruct bootstrapped 52K instruction–response pairs from 175 hand-written seeds by asking a model to write new tasks in the style of the seeds and filtering near-duplicates[12]; Evol-Instruct made the seeds progressively harder — add constraints, deepen, concretize, complicate the reasoning — to manufacture the complexity that Chapter 11's Deita results say is what SFT data is for[15]; and Magpie removed the seeds altogether, exploiting the fact that a chat-tuned model prompted with only its user-turn template will *generate a user query*, then answer it — four million pairs from nothing, and a base model fine-tuned on them matched the official instruct model on AlpacaEval[13]. The persona approach seeds all of these with a who: "as a Peruvian tax accountant, write a request you would make of an assistant" produces a different query distribution from the same prompt without the persona, and a billion of them produce coverage no seed set can[6]. Tülu 3's post-training set used exactly this persona-driven generation for the skills (math, code, precise instruction following) where open prompts were thin[16].

Preferences. A preference pair needs two responses and a judgment. The judged route — sample several responses from several models, score them with a strong judge on a rubric, pair the best against a worse one — is UltraFeedback's recipe (64k prompts × 4 responses, GPT-4 scores)[17] and inherits every judge bias from Chapter 11: verbosity, position,

self-preference. The *on-policy* route, which Llama 3 used at scale, samples the responses from the model being trained, so the pairs are about the mistakes it actually makes, and re-generates them each round as the policy moves[18]. For a verifiable domain the judgment is free (correct vs. incorrect samples of the same prompt), and for everything else the margin filter of Chapter 11 is the first thing to apply.

Reasoning traces, and rejection sampling as the quality station. The pattern that produced DeepSeek-R1's 800K-example SFT set, OpenThoughts3's 1.2M, and Llama 3's post-training data is the same: sample N responses per prompt, verify or judge each, keep the ones that pass, train on them[14][18]. Rejection sampling fine-tuning (RFT) and STaR made the mechanism explicit — the model's own correct reasoning, filtered by an answer check, is a better SFT target than a human-written solution, and more samples per prompt keep helping because distinct correct paths are distinct training signal[19]. The arithmetic the station runs on is pass@k: if the generator solves a prompt with probability p per sample, the chance that k samples contain at least one correct trace is

$$\text{pass@}k \;=\; 1-(1-p)^{k}, \qquad E[\text{kept traces}] = k\,p,$$

so for a prompt at $p = 0.3$, eight samples yield a correct trace 94% of the time and about 2.4 kept traces; for a prompt at $p = 0.02$, eight samples find one 15% of the time, and sixty-four find one 73% of the time at thirty times the cost. The station's budgeting decision is therefore per-prompt, and it is the same curriculum as Chapter 11's pass-rate bins: spend samples where p is in the middle, give up on the bottom until the model improves, and do not bother with the top. Figure 27.5 draws the funnel with the oversampling ratio that production pipelines converge on — generate roughly three times what you will keep.

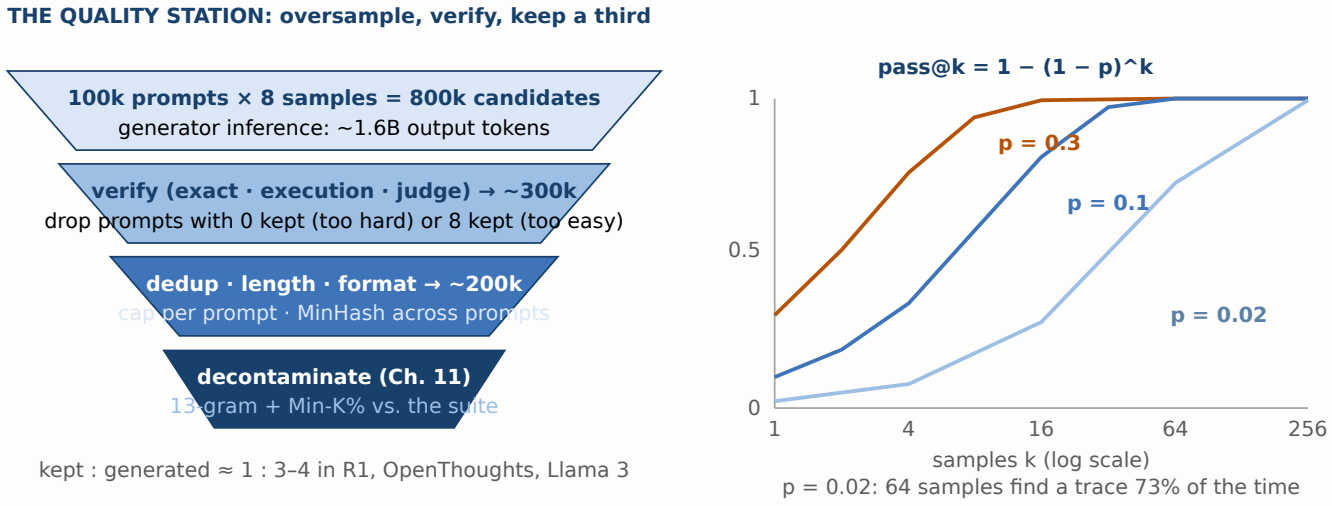


Figure 27.5 — Quality is made by what you throw away. Left: the funnel every published reasoning set went through; the inference bill is for the top, not the bottom. Right: pass@k tells you where extra samples buy traces and where they buy nothing — the same bins Chapter 11 used to measure marginal value.

Two refinements that the published pipelines agree on. *Cap traces per prompt* (R1 kept several correct traces per prompt but filtered the unreadable; OpenThoughts found 16× more data per question from multiple traces, and that the gain was from scale, not from any verification method[14]). And *verify what has a verifier, judge the rest, and keep the two populations separate in the lineage record*: a judged trace is a different kind of evidence from an executed one, and Chapter 11's calibration sample applies to the judged half.

```
# rejection sampling with a verifier; per-prompt budget from pass-rate bins (Ch.
11)
from collections import defaultdict
def rejection_sample(prompts, gen, verify, k_by_bin={"mid": 8, "hard": 32, "easy":
2}):
    kept = defaultdict(list)
    for p in prompts:
        k = k_by_bin[p["bin"]]                        # bins from a pilot pass-rate
run
        sp = SamplingParams(temperature=1.0, max_tokens=8192)
        traces = [o.outputs[0].text for o in gen.generate([p["text"]] * k, sp)]
        ok = [t for t in traces if verify(p, t)]      # exact match / execution /
judge
        if 0 < len(ok) < k:                           # skip all-fail and all-pass
prompts
            kept[p["id"]] = sorted(set(ok), key=len)[:4]   # dedup, cap, prefer
concise
    return kept
# then: MinHash across prompts (Ch. 10); 13-gram + Min-K% vs. the suite (Ch. 11);
log k
```

27.7 Model collapse, and the version that is not

You will be asked about the *Nature* paper in every review: models trained on the outputs of models degrade generation over generation, the tails of the distribution vanish first, and after a few rounds the text is fluent nonsense[20]. The result is real and the mechanism is worth understanding, because it tells you exactly which pipelines are at risk. Each generation samples a finite set from the previous model and fits to it; sampling loses the rare events, fitting a finite-capacity model loses more, and the recursion compounds. In the simplest tractable version — estimating a Gaussian's variance from M samples of the previous generation's estimate — the expected variance after n rounds shrinks geometrically,

$$E\left[\sigma_n^2\right] \;=\; \sigma_0^2\left(1-\tfrac{1}{M}\right)^n,$$

and a language model's long tail of rare constructions, names, and facts is the variance that goes first. But the premise is *replacement*: each generation trains only on the previous generation's output and discards the original data. Under *accumulation* — every generation trains on all the real data plus all the synthetic data so far — the same analysis gives a test error bounded by a constant independent of the number of generations, and the empirical runs across language models, diffusion models, and VAEs show no collapse[21]. Figure 27.6 contrasts the two.

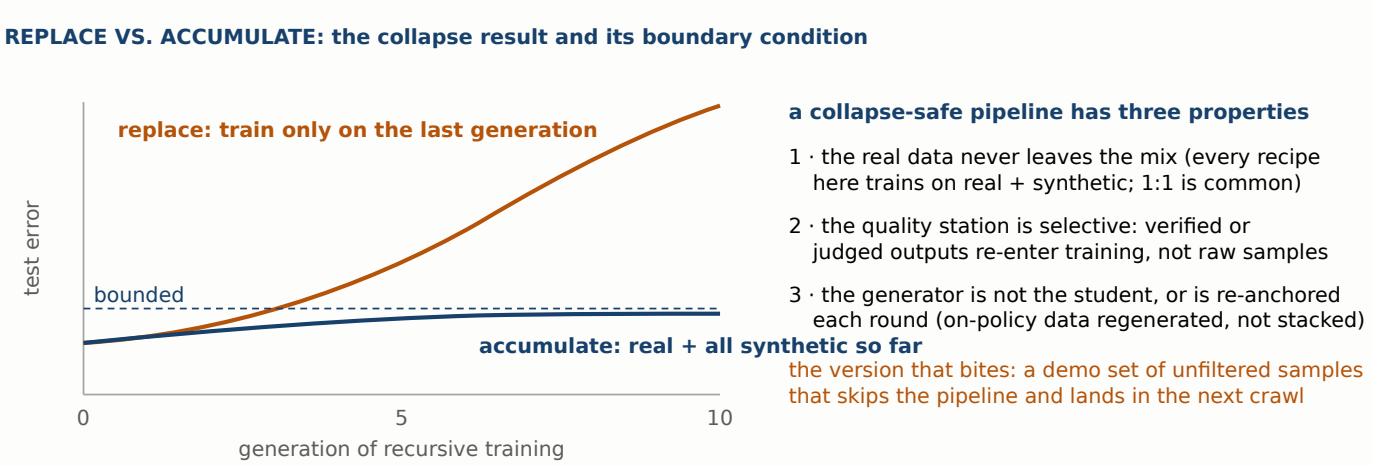


Figure 27.6 — Collapse needs replacement. The published collapse experiments discard the real data each round; the accumulation experiments keep it and stay bounded. Every production recipe in this chapter accumulates, verifies, and mixes — which is why the frontier trains on synthetic data without collapsing.

The risk that *does* apply to you is the quiet one. Synthetic text leaks: into the next Common Crawl, into the open datasets you will download next year, into your own corpus by way of a demo set that skipped the pipeline. Chapter 10's lineage record is the defense (a document whose source is a generator is marked as such and is never counted as real data for the accumulation ratio), and Chapter 11's membership tests are the audit. The **compliance station** adds two more checks that are gates, not suggestions: the license of the generator (several frontier models prohibit using outputs to train competitors; Nemotron-4's kit was released permissively for exactly this reason[2]), and the provenance of the seeds, since a rephrased copyrighted document is still that document in a different form. And since the generator has seen the benchmarks, synthetic data is the most likely place for a contamination leak to enter — run Chapter 11's three tests on the synthetic set *before* mixing it in, not after training.

27.8 What the big labs do

Table 27.2 — Synthetic data in shipped pipelines. Scale, generation pattern, and the quality station, as disclosed.

RECIPE	SCALE	GENERATION	QUALITY STATION
Phi-4[1]	~400B pre-training tokens; most of post-training	~50 generator types: textbooks, Q&A, code, seeded from web and books; multi-agent and self-revision prompts	Execution and verification for code/math; rewriting loops; decontamination with the model's own eval suite
Nemotron-CC[3]	1.9T synthetic of 6.3T total	Rephrase LQ crawl to usable text; HQ crawl to diverse QA, summaries, knowledge lists	Classifier ensemble on source; dedup; the synthetic part is what made LQ crawl trainable (+5.6 MMLU vs. DCLM)
Qwen3[4]	Trillions of tokens of 36T	Qwen2.5-Math and Qwen2.5-Coder specialists generate	Specialist models as both generator and filter; staged into the reasoning

RECIPE	SCALE	GENERATION	QUALITY STATION
		textbooks, QA, and code; PDF text via Qwen2.5-VL	stage of the curriculum
Meta WikiExpert[9]	1T Active Reading tokens from 6M articles	Self-written study strategies applied per document; same 8B model	Mixed 1:1 with real data over 4 epochs; guardrail benchmarks monitored
Llama 3[18]	Millions of post-training examples per round	On-policy rejection sampling; execution-feedback loops for code; translation and long-context tasks	Reward model and verifier filtering; several rounds, regenerated each time
DeepSeek-R1[14]	~600K reasoning + ~200K general	Sampled from the RL checkpoint; general data from DeepSeek-V3	Rule-based verification; readability filter; generative reward model for the rest
Tülu 3[16]	~940K SFT prompts, a large synthetic share	Persona-driven generation for math, code, and precise instruction following	Verifiable rewards for RLVR; aggressive decontamination; development vs. unseen suites

27.9 Practical considerations

Cost the factory like a serving problem. Generation is inference, so Chapter 13's economics apply: batch it, use a quantized generator with speculative decoding, run it on the cheapest hardware that holds the model, and schedule it in the gaps of the training cluster. The FinePhrase rate — about 9,200 tokens per second per GPU for a 1.7B generator — is the number to beat; at that rate a 7B generator without speculation is several times slower, and a 27B one an order of magnitude, for gains the ablations do not find.

Pilot every recipe on a small student before scaling the job. A 300M–1B student trained on 10–20B tokens of a candidate recipe (half synthetic, half real) is a few hundred GPU-hours and separates good prompts from bad with the same resolution the published studies used. Fit the §27.5 law on the pilot; do not generate a trillion tokens of a recipe you have not measured.

Spanish and Portuguese are a synthesis problem first. In-language rephrasing of the in-language crawl (FineWeb2's Spanish and Portuguese subsets) with a generator that writes the regional register well, Active Reading over the regional corpora that exist in small volume, and persona-seeded instruction generation with Latin American personas are the three levers; translation of English synthetic sets is the fallback, and its outputs should be audited with Chapter 11's human sample for register and for the judge's English bias. Re-measure the share after tokenization, as Chapter 12 will insist.

Write the generator into the dataset card. Model, version, prompt version, sampling parameters, verifier, and the kept-to-generated ratio, per shard. Chapter 10's lineage record has a field for each; when a result cannot be reproduced a

year later, the missing field is usually the prompt version.

Common pitfalls

Generating more of the same.

Fluent, on-distribution, and worth less than the source. The value is in the transformation; if the prompt does not change the form, the data will not beat the web.

Scaling the generator before the prompt.

Prompt design dominated every ablation; 1B–3B generators captured the gains; 27B generators cost 5–10× more and often scored lower.

Training on synthetic data alone.

Every study lost general-capability points without the real mix-in; the trillion-token knowledge model needed it to keep its guardrail benchmarks. Keep the real data in, at roughly 1:1.

Skipping the quality station because the generator is strong.

Kept-to-generated ratios of 1:3 are the norm in shipped pipelines. A strong generator produces plausible wrong answers; only a verifier or a calibrated judge tells them apart.

Letting synthetic data skip the contamination ring.

The generator has seen the benchmarks. Run the Chapter 11 tests on the synthetic set before it is mixed in.

Fearing collapse in the wrong place.

Collapse needs replacement; your accumulating, verified, mixed pipeline is not at risk. Your demo set that skipped the pipeline and landed in a public repository is.

Measuring only the target.

Active Reading raised SimpleQA and lowered GSM8K; a synthetic math set can raise MATH and lower chat quality. Chapter 9's full suite, every time.

27.10 Decision framework

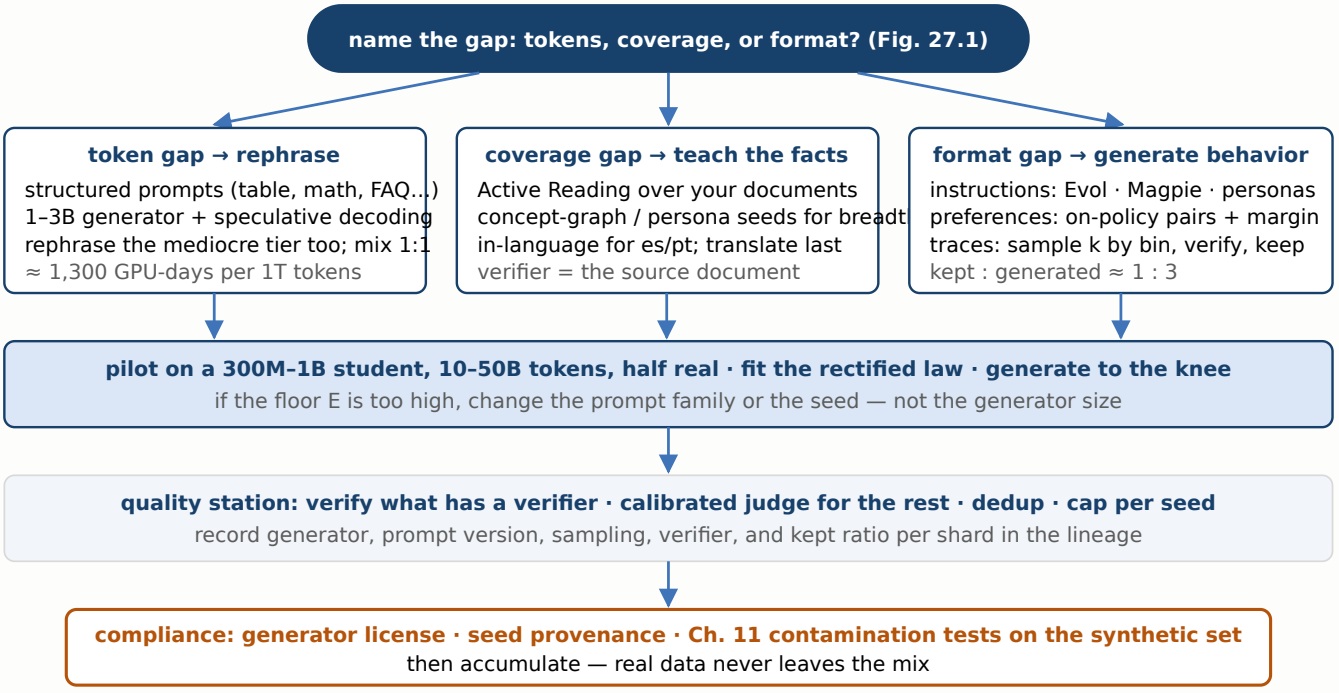


Figure 27.7 — Choosing a synthesis recipe. The gap picks the generation pattern; the pilot and the rectified law pick the volume; the quality and compliance stations are the same for all three. Every path ends in accumulation, which is what makes it collapse-safe.

When it hurts. Synthesis hurts when the generator does not know the domain (it will produce confident, verifiable-looking errors; a verifier or the source document is non-negotiable); when the recipe is past its knee (the law says more tokens of the same kind do nothing — change the prompt family); and when the mix drifts synthetic-heavy (general capabilities slide; the real mix-in restores them). It also hurts quietly when it works: a model trained on a trillion synthetic Wikipedia tokens is better at facts and slightly worse at arithmetic, and a committee that sees only the target benchmark will not know.

How this scales. Generation cost scales with the student's token budget, not with the student's size, so the synthetic fraction of the bill falls as models grow — a quarter of training cost for the 7B/2T example, a tenth at 70B — while the data wall that motivates it rises. The one quantity that moves the other way is the floor E: a larger student exhausts a recipe sooner (SynthLLM's 8B at 1T vs. 3B at 4T), so larger models need *more diverse* generation, not more of it.

27.11 Summary

Synthetic data is a factory with three stations, and the value is made at the second one. Name the gap first: a token gap is closed by rephrasing the web into forms the web lacks — tables, problem sets, tutorials, conversations — with a 1–3B generator, structured prompts, and a one-to-one mix with real text, which the 2025–2026 studies show beating the open web by several points at a fraction of the training tokens; a coverage gap is closed by teaching facts in many forms, where Active Reading's self-written study strategies took an 8B model from 7% to 66% on closed-book questions about its documents; a format gap is closed by generating the behavior and keeping a third of it after

verification. Make as much as the rectified law says — synthetic tokens have a floor set by the generator's diversity, measured on a small pilot — and no more of the same kind. Verify what can be verified, calibrate the judge for the rest, decontaminate before mixing, record the generator in the lineage, and never let the real data leave the mix: that last rule is the whole difference between the frontier's pipelines and the collapse experiments. For the teams whose real-data constraint binds hardest — small languages, specialized domains, modest budgets — the factory is the great equalizer, and it rewards the discipline this book keeps preaching: measure the funnel, verify what you can, and never let the demo data skip the pipeline.

References

1. Abdin, M., et al. (2024). Phi-4 Technical Report. arXiv:2412.08905. ~400B tokens of curated synthetic pre-training data from ~50 generator types; synthetic-heavy post-training.
2. NVIDIA (2024). Nemotron-4 340B Technical Report. arXiv:2406.11704. 98% synthetic alignment data; generator, judge, and preference models released under a permissive license.
3. Su, D., et al. (2024). Nemotron-CC: Transforming Common Crawl into a Refined Long-Horizon Pretraining Dataset. arXiv:2412.02595. 6.3T tokens of which 1.9T synthetic; rephrasing of low- and high-quality crawl; +5.6 MMLU over DCLM at 1T tokens on 8B models.
4. Qwen Team (2025). Qwen3 Technical Report. arXiv:2505.09388. 36T tokens; trillions of synthetic tokens from Qwen2.5-Math and Qwen2.5-Coder; PDF extraction with Qwen2.5-VL.
5. Maini, P., et al. (2025). BeyondWeb: Lessons from Scaling Synthetic Data for Trillion-scale Pretraining. DatologyAI report, arXiv:2508.10975. +5.1 over Cosmopedia and +2.6 over Nemotron-Synth at 8B on 14 benchmarks; 7.7× faster than open web; 3B on BeyondWeb beats 8B on Cosmopedia; generator-size ablation 1B +1.8 / 3B +3.3 / 8B +3.7; conversational share saturates near 20%.
6. Ge, T., et al. (2024). Scaling Synthetic Data Creation with 1,000,000,000 Personas. arXiv:2406.20094. Persona Hub; 1.07M generated math problems; 64.9% MATH for a fine-tuned 7B model.
7. Niklaus, J., et al. (2026). How Can We Synthesize High-Quality Pretraining Data? A Systematic Study of Prompt Design, Generator Model, and Source Data (FinePhrase). Hugging Face, arXiv:2604.13977. 1T+ generated tokens across 12 prompts, 6 generator sizes, 6 families; structured formats (math 15.31 vs. DCLM 13.77); Gemma-3 1B beats 27B; always mix with real text; FinePhrase-Table 486B tokens at ~9,200 tok/s/GPU (~612 GPU-days), +3.41 over DCLM.
8. Maini, P., Seto, S., Bai, H., Grangier, D., Zhang, Y., & Jaitly, N. (2024). Rephrasing the Web: A Recipe for Compute and Data-Efficient Language Modeling (WRAP). *ACL*. arXiv:2401.16380. ~3× pre-training speed-up; >10% perplexity improvement; >2% zero-shot QA gain.
9. Lin, J., et al. (2025). Learning Facts at Scale with Active Reading. Meta FAIR; *ICLR 2026*. arXiv:2508.09494. Self-generated document-specific study strategies; SimpleWikiQA 66.3 vs. 25.7 paraphrase and 47.9 synthetic QA at 8B; Meta WikiExpert-8B on 1T Active Reading tokens: SimpleQA 23.5 vs. 7.3 base; self-generated data beat 70B-generated (66.3 vs. 62.3); guardrail regressions on GSM8K and MBPP.

10. Ben Allal, L., et al. (2024). Cosmopedia. Hugging Face release and report. 25B tokens, 30M textbook-style documents generated by Mixtral-8x7B.

11. Qin, Z., et al. (2025). Scaling Laws of Synthetic Data for Language Models (SynthLLM). arXiv:2503.19551. Concept extraction and graph-based recombination; rectified scaling law; plateau near 300B tokens; 8B peaks near 1T vs. 3B near 4T.

12. Wang, Y., et al. (2023). Self-Instruct: Aligning Language Models with Self-Generated Instructions. *ACL*. arXiv:2212.10560. 175 seeds → 52K instructions.

13. Xu, Z., et al. (2024). Magpie: Alignment Data Synthesis from Scratch by Prompting Aligned LLMs with Nothing. arXiv:2406.08464. 4M instruction pairs from the chat template alone; fine-tuned base matches the official instruct model on AlpacaEval.

14. DeepSeek-AI (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. ~600K reasoning and ~200K non-reasoning rejection-sampled SFT examples; readability filtering. Guha, E., et al. (2025). OpenThoughts: Data Recipes for Reasoning Models. arXiv:2506.04178. Multiple traces per question; verification without significant gains at 1.2M.

15. Xu, C., et al. (2024). WizardLM: Empowering Large Language Models to Follow Complex Instructions (Evol-Instruct). *ICLR*. arXiv:2304.12244.

16. Lambert, N., et al. (2024). Tülu 3: Pushing Frontiers in Open Language Model Post-Training. arXiv:2411.15124. Persona-driven synthetic prompts for math, code, and precise instruction following; RLVR; decontamination toolkit.

17. Cui, G., et al. (2024). UltraFeedback: Boosting Language Models with Scaled AI Feedback. *ICML*. arXiv:2310.01377. 64k prompts × 4 responses with GPT-4 scores.

18. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783. On-policy rejection sampling over several post-training rounds; execution feedback for code; reward-model filtering.

19. Yuan, Z., et al. (2023). Scaling Relationship on Learning Mathematical Reasoning with Large Language Models (RFT). arXiv:2308.01825. Rejection-sampling fine-tuning; more distinct correct reasoning paths keep improving. Zelikman, E., et al. (2022). STaR: Bootstrapping Reasoning with Reasoning. *NeurIPS*. arXiv:2203.14465.

20. Shumailov, I., et al. (2024). AI Models Collapse When Trained on Recursively Generated Data. *Nature* 631, 755–759. Tail-first degradation under replace-style recursive training.

21. Gerstgrasser, M., et al. (2024). Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data. arXiv:2404.01413. Accumulation gives a test error bounded independently of the number of generations; no collapse across LMs, diffusion models, and VAEs.